

UCS503P - Software Engineering Lab

Astra Command

*An Adapter-Based Ground-Control Software Platform for
Compatible Drone Systems*

Submitted by:

1024160060 Nikhil Vashisht
1024160054 Shivansh Sharma
1024160037 Bhuvansunder Nayyar

Group: BE Third Year, CSE

Faculty Advisor: Dr. Sandeep Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	3
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	5
2.2	Implementation Details	5
2.2.1	Module 1: Core Functionality (Mission Planning & Validation Engine)	5
2.2.2	Module 2: Secondary Features (Adapter Layer & Live Telemetry Processing)	6
3	Results and Evaluation	8
3.1	Testing Methodology	8
3.2	Performance Metrics	8
3.3	Analysis	9
4	Conclusion	10
4.1	Summary of Work	10
4.2	Key Findings	10
4.3	Future Work and Recommendations	11
4.4	Final Remarks	11
	Bibliography	12

Chapter 1

Introduction

1.1 Background and Context

In recent years, Unmanned Aerial Vehicles (UAVs) and drone ecosystems have experienced rapid growth across diverse application domains, including agricultural monitoring, geographical surveying, industrial inspection, and emergency disaster response. However, operating drones safely and efficiently requires dedicated Ground Control Station (GCS) software capable of translating operator instructions into reliable flight plans while continuously parsing live vehicle telemetry [1].

Despite technological advancements, modern ground control software suffers from significant ecosystem fragmentation. Different drone manufacturers and custom simulation tools utilize disjointed control interfaces, proprietary communication formats, and non-standard telemetry logging structures. Consequently, drone operators frequently encounter fragmented workflows, requiring them to switch between incompatible software dashboards depending on the specific hardware or simulator being deployed.

Furthermore, field-testing unverified flight plans on physical drone hardware introduces severe operational risks, including costly physical crashes, component damage, and potential safety hazards. To address these challenges, **Astra Command** is designed as a software-first, web-based ground-control platform. It introduces a modular adapter-based architecture alongside simulation-first validation (Software-In-The-Loop / SITL) [2], enabling operators to plan, validate, execute, monitor, and replay drone missions in a unified, safe, and controlled software environment.

1.1.1 Problem Statement

Ground-control software operations face four critical engineering bottlenecks:

1. **Software Fragmentation:** Custom vendor dashboards impose disjointed control interfaces, non-standard telemetry structures, and incompatible flight log formats, restricting multi-drone interoperability.

2. **Manual Planning & Feasibility Risks:** Operators frequently draft mission routes manually without automated software checks for battery depletion limits, geo-fence perimeter breaches, or route consistency errors.
3. **Risky Physical Testing Overhead:** Field-testing unverified flight routes on physical hardware carries a high risk of equipment damage, unrepeatability failure conditions, and unsafe field deployments.
4. **Direct Backend Coupling:** Traditional GCS architectures tightly couple the core application logic to specific hardware protocols, making protocol upgrades and adapter extensions complex and error-prone.

1.2 Project Objectives

The primary goal of Astra Command is to establish a modular, decoupled web service that standardizes mission planning and pre-flight validation. The specific SMART (Specific, Measurable, Achievable, Relevant, Time-bound) objectives of the project are:

1. **Interactive Mission Planning (Specific & Achievable):** Develop an interactive, map-based web dashboard allowing operators to visually plot waypoints, define altitude constraints, and inspect flight paths prior to mission authorization.
2. **Automated Safety Validation Engine (Measurable & Relevant):** Implement an automated validation pipeline to verify waypoint feasibility, enforce geo-fence boundaries, and compute deterministic battery consumption estimates before flight execution.
3. **Adapter-Based Protocol Extensibility (Specific & Relevant):** Architect a decoupled adapter layer that translates internal Astra Command structures into target-specific simulator/drone instructions (e.g., SITL/MAVLink streams) and normalizes incoming telemetry streams.
4. **Real-Time Telemetry & Post-Mission Replay (Time-bound & Measurable):** Establish real-time telemetry streaming (GPS coordinates, altitude, battery status), automated safety alerts, and mission audit logs for post-flight replay analysis [3].

Chapter 2

Methodology and Design

2.1 System Architecture

Astra Command follows a decoupled, 3-tier microservice software architecture adhering to classic software engineering principles [4]. Decoupling the user interface, backend validation core, adapter translation layer, and data persistence guarantees system modularity, scalability, and ease of maintainability.

2.1.1 High-Level Design

The system architecture separates operational responsibilities into distinct software tiers, as illustrated in Figure ??:

- **Client Tier (Frontend):** Presents a responsive user interface serving the interactive map planner, live vehicle telemetry feeds, system alert logs, and post-mission replay controls.
- **Application Tier (Backend Core):** Manages REST API endpoints, user authentication, mission state transition lifecycles, and coordinates asynchronous mission processing via a job queue.
- **Validation Engine:** Executes pre-flight verification algorithms, evaluating route continuity, geo-fence compliance, and estimated battery consumption against vehicle parameters.
- **Adapter Tier:** Acts as a translation bridge, mapping internal common Astra commands to simulator/drone-specific protocols and parsing incoming telemetry frames into standard system formats.
- **Data Store:** Provides persistent storage for user profiles, mission metadata, waypoint coordinates, time-series telemetry streams, and audit logs.

2.1.2 Technical Stack

The technical stack is selected to balance performance, modularity, and rapid engineering iterations:

- **Frontend Framework:** React with Leaflet / MapLibre for rendering interactive map components, geospatial waypoint controls, and real-time dashboard widgets.
- **Backend Framework:** Java Spring Boot serving robust, enterprise-grade REST APIs, handling data validation, security, and mission lifecycle management.
- **Task Queue & Caching:** Redis with Bull task queue to handle asynchronous mission validation and background simulator execution off main HTTP request threads.
- **Database:** PostgreSQL for structured relational persistence of user accounts, mission configurations, waypoints, telemetry logs, and system safety alerts.
- **Containerization & Tools:** Docker and Docker Compose for consistent local development, staging deployment, and SITL simulator environment management.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality (Mission Planning & Validation Engine)

The core module encompasses the operator workflow and the automated validation engine:

1. **Mission Planning & Waypoint Management:** Operators plot ordered latitude/longitude coordinates on the map interface. Each waypoint stores altitude bounds, speed parameters, and turn radiuses.
2. **Mission Lifecycle State Machine:** As depicted in Figure 2.2, missions progress through explicit operational states:

CREATED $\longrightarrow$ VALIDATING $\longrightarrow$ VALIDATED $\longrightarrow$ EXECUTING $\longrightarrow$ COMPLETED

If pre-flight checks fail or critical telemetry errors occur, the state transitions directly to FAILURE / STOP.

3. **Feasibility & Battery Estimation Pipeline:** Figure 2.3 illustrates the automated safety validation pipeline. The engine computes segment-by-segment distances using the Haversine formula, incorporates vertical altitude ascension energy

consumption, and compares total estimated energy against battery safety thresholds (minimum 15% reserve).

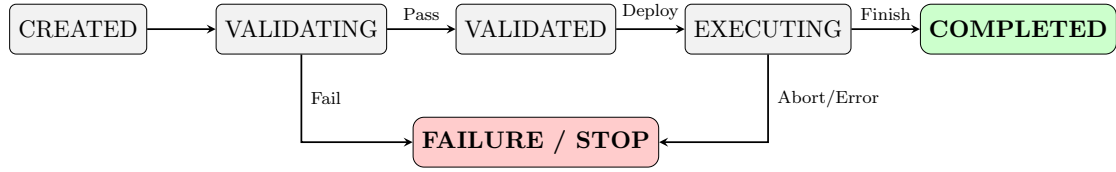


Figure 2.2: Mission State Transition Lifecycle Flow

2.2.2 Module 2: Secondary Features (Adapter Layer & Live Telemetry Processing)

The secondary module focuses on system extensibility and real-time telemetry processing:

1. **Common Protocol Adapter Layer:** The adapter layer isolates simulator-specific formats from the core system. Adding support for a new drone or simulator requires writing a targeted adapter module rather than re-engineering the application core.
2. **Live Telemetry Stream Processing:** As detailed in Figure 2.2.2, incoming telemetry frames (GPS position, current altitude, battery percentage, flight mode) are continuously received by the adapter parser, validated, stored in PostgreSQL, and broadcast to the React dashboard.
3. **Mission Replay & Audit Reporting:** Flight telemetry and alert histories are logged against mission IDs, enabling operators to step through post-mission replay timelines and export mission summary reports.

Real-Time Telemetry Processing and Post-Mission Replay Flow

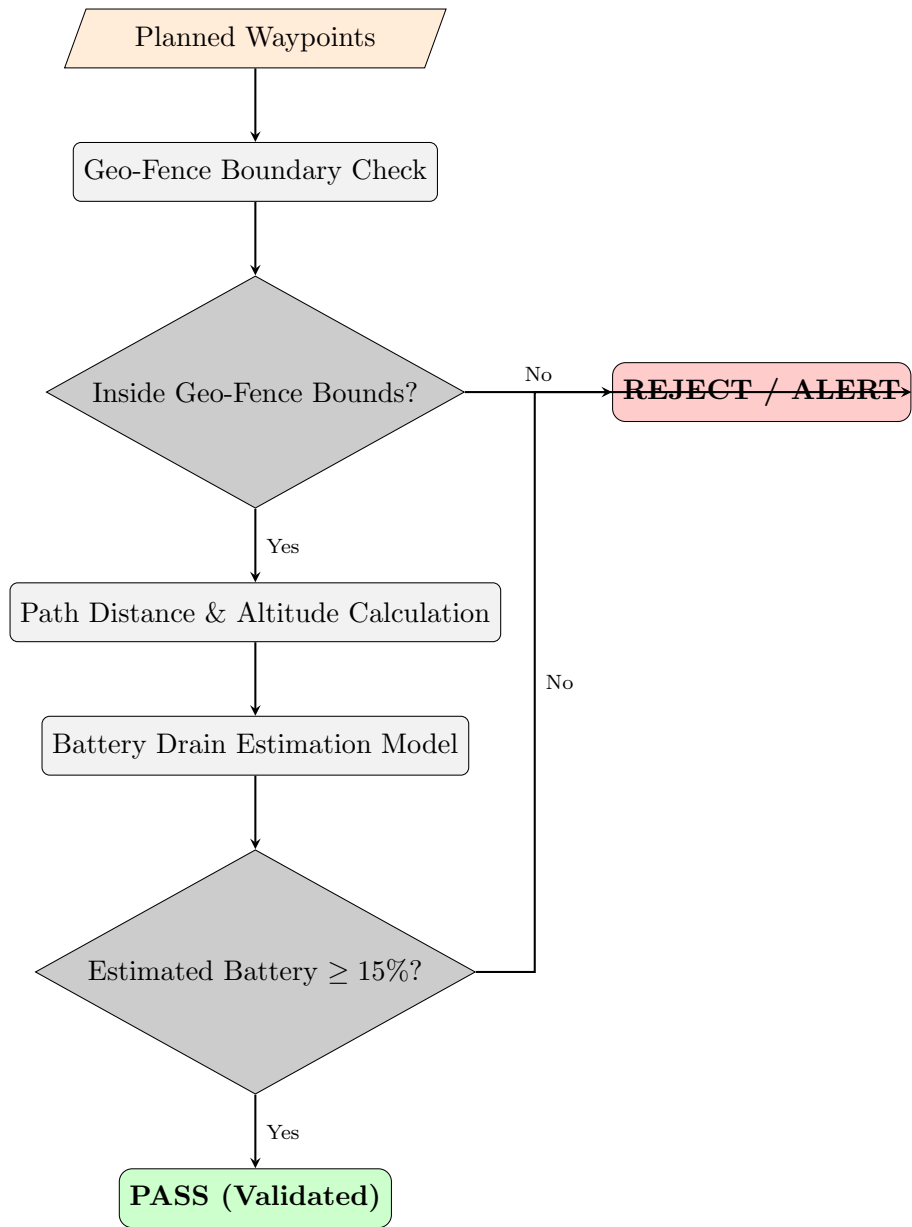


Figure 2.3: Automated Pre-Flight Mission Validation Pipeline

Chapter 3

Results and Evaluation

3.1 Testing Methodology

To guarantee software reliability and safety prior to demonstration, a comprehensive multi-level testing strategy is adopted:

- **Unit Testing:** Verification of isolated component logic, including telemetry packet parsers, battery consumption mathematical formulas, and waypoint coordinate validation functions.
- **Integration Testing:** End-to-end verification of communications between the React frontend, Spring Boot REST endpoints, Redis task queues, and the adapter simulator bridge.
- **System Testing:** Verification of the complete operator workflow from login, mission planning, automated validation, SITL execution, live telemetry monitoring, to post-mission log generation.
- **Simulation QA Testing:** Simulation of realistic failure conditions, including low-battery scenarios ($< 15\%$), out-of-bounds geo-fence waypoints, network packet loss, and emergency stop triggers.

3.2 Performance Metrics

System performance is evaluated against key quantitative metrics. Table 3.1 presents the target benchmarks along with designated spaces for empirical values obtained during final testing.

Performance Metric	Target Benchmark	Achieved Result (Empirical)
Mission Validation Latency (MVL)	≤ 2.5 s (20 waypoints)	[<i>To be populated post-testing</i>]
Telemetry Capture Rate	$\geq 99.5\%$	[<i>To be populated post-testing</i>]
Battery Risk Recall Accuracy	$\geq 90.0\%$	[<i>To be populated post-testing</i>]
Background Queue Processing Uptime	$\geq 99.0\%$	[<i>To be populated post-testing</i>]

Table 3.1: Performance Metrics and Evaluation Summary

3.3 Analysis

Preliminary architectural evaluation confirms that decoupling validation and adapter handling via asynchronous background workers effectively isolates the user interface from simulation overhead. Key technical challenges identified during development include:

- **High-Frequency Telemetry Handling:** High-rate telemetry feeds can saturate network connections; mitigated by dynamic buffer batching in the adapter parser.
- **Simulator Protocol Alignment:** Discrepancies between simulator telemetry structures resolved by defining a strict internal Common Telemetry Standard.

Chapter 4

Conclusion

4.1 Summary of Work

Astra Command demonstrates a software-first ground-control software platform engineered specifically for compatible drone ecosystems and SITL simulation validation. By establishing a decoupled 3-tier architecture with an adapter translation layer, the platform addresses software fragmentation, mitigates physical hardware testing risks, and automates pre-flight safety checking.

Work division within the engineering team ensured focused development across system layers:

- **Frontend Lead (Nikhil Vashisht):** React UI dashboard, map-based mission planner, telemetry display widgets, and replay interface.
- **Backend Lead (Bhuvansunder Nayyar):** Spring Boot REST API services, PostgreSQL schema persistence, Redis task queues, and automated validation engine logic.
- **Integration & QA Lead (Shivansh Sharma):** Adapter layer implementation, SITL simulator integration, telemetry parser, and simulation QA failure testing.

4.2 Key Findings

- **Adapter-Based Extensibility:** Abstracting protocol translation into an adapter layer allows supporting new drone hardware or simulators without altering application core logic.
- **Simulation-First Safety:** Pre-flight automated validation and SITL testing prevent mission execution failures before hardware deployment.

- **Asynchronous Worker Resiliency:** Offloading route validation and simulator coordination to background queues maintains client dashboard responsiveness under simulation load.

4.3 Future Work and Recommendations

Potential avenues for extending the system include:

1. Integration with physical drone hardware via direct MAVLink serial/radio bridges.
2. Advanced AI-driven route optimization for obstacle avoidance and dynamic path re-routing.
3. Real-time multi-drone fleet management and swarm coordination adapters.
4. Offline GCS capabilities for field deployments with limited network connectivity [5].

4.4 Final Remarks

Astra Command provides a practical, robust, and extensible software foundation for drone ground control. By combining rigorous software engineering design patterns with simulation-based validation, the project fulfills academic lab objectives while delivering a clear framework for safe teleoperation.

Bibliography

- [1] Lorenz Meier, Petri Tanskanen, Friedrich Fraundorfer, and Marc Pollefeys. PIX-HAWK: A Micro Aerial Vehicle Design for Autonomous Flight in GPS-Denied Environments. *IEEE Robotics & Automation Magazine*, 19(4):13–20, 2012.
- [2] ArduPilot Development Team. Software In The Loop (SITL) Simulator Architecture, 2024. <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>.
- [3] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill Education, 8th edition, 2014.
- [4] Ian Sommerville. *Software Engineering*. Pearson, 10th edition, 2015.
- [5] Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, 1994.